



go through sih website check problem statement 228 and tell some repos on git which provides bold and unique solution for it

Based on the current Smart India Hackathon (SIH) 2026 Master Catalogue, Problem Statement **228** corresponds to **SIH26233: Inline Microbial Contamination Detection Using Hyperspectral Edge Sensors**.

This is a high-priority challenge from the **Ministry of Food Processing Industries (MoFPI)**. The objective is to build a conveyor-mounted Hyperspectral Imaging (HSI) rig equipped with embedded Edge AI computing to detect microbial contamination (like *E. coli*) on food items in milliseconds.

To win this challenge, you need an architecture that seamlessly balances complex hyperspectral data processing with ultra-low latency edge computing. Here are some GitHub repositories that provide bold, unique, and highly advanced solutions you can adapt for your prototype:

1. The Real-Time Hyperspectral Pipeline

Repo: `cubert-hyperspectral/cuvis-ai`

Why it's a game-changer: Dealing with live hyperspectral video on a conveyor belt is incredibly difficult due to the massive size of the data cubes. Cuvis.AI is a brand-new open-source framework designed specifically for building modular AI pipelines that process, analyze, and deploy hyperspectral video data in real-time.

How to use it: Use this as the core backbone of your software architecture. It handles the data ingestion from the HSI hardware and orchestrates the inference layer, allowing your team to focus entirely on the AI model rather than the complex I/O pipelines.

2. Anomaly Detection for Millisecond Edge Inference

Repo: `NomanShahRaza/hyperspectral_anomaly_detection_algorithms`

Why it's a game-changer: Many teams will try to use heavy Convolutional Neural Networks (CNNs) for this, which will lag and fail the "millisecond" requirement on edge devices (like an NVIDIA Jetson Orin). You can take a much smarter approach by framing microbial presence as an *anomaly detection* problem.

How to use it: This repository contains implementations of lightweight, mathematically rigorous algorithms like Global/Local RX and Laplacian Anomaly Detectors (LAD). Implementing these mathematical models instead of heavy neural networks will give you blindingly fast, low-latency edge inference capable of catching *E. coli* at high conveyor speeds.

3. The "Wow" Factor: Mamba Architecture for Hyperspectral Data

Repo: `RsAI-Lab/HDMba` (Hyperspectral Dehazing Mamba)

Why it's a game-changer: If you do want to use deep learning, Vision Transformers (ViTs) are typically the gold standard for hyperspectral images, but they are too heavy for edge devices due to their quadratic complexity [$O(N^2)$]. This repository applies the cutting-edge **Mamba (State Space Model)** architecture to hyperspectral imaging. Mamba allows for long-range spatial-spectral dependencies but operates in *linear complexity* [$O(N)$].

How to use it: While this specific repo is tuned for image dehazing, you can strip out the `DehazeMamba` blocks and adapt the base architecture for semantic segmentation (classifying healthy vs. contaminated pixels). Proposing a Mamba-based Edge architecture will stand out as highly innovative to the SIH judges.

4. Microbial Classification & Explainability (XAI)

Repo: `majsylw/microbial-counting-review` (BacXception) & `gauravfs-14/awesome-mechanistic-interpretability`

Why it's a game-changer: Food safety inspectors will not blindly trust a black-box AI that rejects food batches.

How to use it: Leverage resources from the microbial counting review to pre-train your feature extractors based on actual bacteria morphologies. Then, integrate mechanistic interpretability (Explainable AI) to output a heat map that clearly highlights the exact spectral bands and pixel regions that triggered the contamination alert.



Winning Strategy for SIH:

Don't just build a software model. Because this is a **Hardware Edition** problem, your pitch must include the physical rig design. Propose a system where:

1. **Hardware:** A low-cost UV/NIR illumination tunnel triggers fluorescence in microbial biofilms.
2. **Processing:** The hyperspectral data is piped through `cuvis-ai` into a Jetson Nano/Orin.
3. **AI:** An anomaly detection algorithm (like Local DM) flags the contamination in less than

This is a Ministry of Defence / Indian Army (DGIS) problem statement, ID 26228, in the Blockchain & Cybersecurity theme. In short, they want a trust and verification layer for AI vision systems. It should check three things: that t

This **Ministry of Defence (DGIS)** problem statement is asking for the holy grail of military AI: a **Zero Trust Machine Learning Pipeline**. In a battlefield context, an adversary doesn't need to destroy a drone if they can subtly alter its vision model to ignore specific camo patterns, or intercept and rewrite its classification outputs before they reach command.

To win this challenge, you cannot pitch standard cybersecurity tools (like basic encryption). You need to pitch an architecture that provides **cryptographic guarantees** at every stage of the ML lifecycle.

Here are the bleeding-edge GitHub repositories you can combine to build a bold, unique, and unified framework that solves all three pillars of the MoD's problem:

Pillar 1: Defending Against Data Poisoning & Tracking Provenance

If training data is poisoned with a "backdoor trigger" (e.g., a specific pixel pattern that forces a misclassification), the resulting model is fundamentally compromised.

Repo: `SCLBD/BackdoorBench`

The Bold Solution: This is a comprehensive, state-of-the-art framework for testing data poisoning attacks and defenses.

How to use it: Instead of assuming data is clean, integrate BackdoorBench's defense algorithms (like Anti-Backdoor Learning or spectral signature detection) as an automated gatekeeper. Before any dataset from external contributors is merged into the military's central training repository, this pipeline automatically scans it for statistical anomalies indicating a poisoning attack.

proceedings.neurips.cc

Repo: [in-toto/in-toto](#)

The Bold Solution: A framework to secure software supply chains.

How to use it: Use in-toto to create cryptographic "attestations" for your data pipeline. Every time a contributor submits data, or a script cleans that data, a cryptographically signed receipt is generated. If data was altered outside this approved pipeline, the attestation fails.

Pillar 2: Verifying Model Integrity (Tamper-Proofing)

Once the model is trained, it exists as a massive file of weights. If an insider or hacker alters a few decimal points in the weights, the model's behavior changes drastically.

Repo: [sigstore/model-transparency](#)

The Bold Solution: Built specifically to bring modern software supply chain security to massive AI models. Standard signing tools fail on 10GB+ neural networks, but this project is designed to hash, sign, and verify ML models component-by-component.

How to use it: When your model finishes training, sign it using this tool. It records the signature to an append-only transparency log (Rekor). Anyone using the model can instantly verify mathematically that the weights match exactly what the MoD originally trained. [Red Hat Emerging Technol...](#)

Repo: [sigstore/model-validation-operator](#)

The Bold Solution: A Kubernetes-native operator for AI deployments.

[Sigstore Blog](#)

How to use it: You can configure a military edge node so that the container orchestrator physically *refuses* to load a model into memory unless the model's `model-transparency` cryptographic signature is verified in real-time.

Pillar 3: Verifiable Inference & Output Authenticity

This is the hardest part: proving that a deployed model on an edge device (like a drone) actually ran the computation properly, and that a man-in-the-middle didn't just alter the output payload (e.g., changing "Enemy Tank" to "Civilian Vehicle").

Repo: `zkonduit/ezkl`

The Bold Solution: This introduces **Zero-Knowledge Machine Learning (zkML)** to your pipeline. `ezkl` is an engine that takes standard neural networks (exported as `.onnx` files) and compiles them into a zk-SNARK circuit. [GitHub](#)

How to use it: When the computer vision model runs an inference on an image, `ezkl` generates both the output *and* a lightweight cryptographic proof. This proof mathematically guarantees: *"I ran this exact specific model, on this specific input image, and it produced this exact output"*.

Why the MoD will love it: Command headquarters can verify the zk-SNARK proof in milliseconds. Even if an enemy completely compromises the edge device operating system, they cannot fake the AI's output without invalidating the cryptographic proof.

💡 The Winning Architecture Pitch for SIH

Don't present these as separate tools. Propose the **"Cryptographic AI Supply Chain"**:

1. **Ingestion:** Data is scanned for backdoors (`BackdoorBench`) and signed (`in-toto`).
2. **Training:** The final model weights are hashed and bound to an immutable transparency log (`model-transparency`).
3. **Deployment:** Kubernetes (`model-validation-operator`) checks the hash before allowing the weapon system to load the AI.

BackdoorBench's defense algorithms (like Anti-Backdoor Learning or spectral signature detection) does military uses this or not currently

No, they are **not** currently used in widespread, deployed operational military systems (like active combat drones, targeting systems, or live intelligence pipelines).

If a military organization buys an AI model from a defense contractor today, they are generally relying on traditional cybersecurity (air-gapping the networks, strict access controls, and background checks on personnel) rather than automated, mathematical anomaly detection inside the AI itself.

However, they are heavily used in **military R&D and testbeds**. Here is the reality of where this technology stands in the defense sector right now:

1. The R&D Phase (Where we are now)

Major defense organizations are desperately researching these exact algorithms because they know their current systems are vulnerable. For example, the US Defense Advanced Research Projects Agency (DARPA) runs a specific program called **GARD (Guaranteeing AI Robustness against Deception)**. Programs like GARD are actively funding academic research into techniques like spectral signatures and Anti-Backdoor Learning to see if they can survive real-world military conditions. [darpa.mil](https://www.darpa.mil)

2. The Operational Reality (What they actually do today)

Currently, if the Indian Army (DGIS) or the US DoD wants to ensure an AI model isn't poisoned, they use a highly manual, brute-force approach:

"Clean Rooms": They try to generate all training data entirely in-house within secure, classified environments so external attackers cannot touch it.

Red Teaming: They pay human cybersecurity teams to try and "break" or trick the model before it is deployed.

The Problem: This manual approach does not scale. Modern AI requires massive amounts of data from open-source intelligence, allied nations, and commercial satellites. They cannot manually vet all of it, leaving a massive security gap.

Why this is your SIH Superpower

This gap is the exact reason **Problem Statement 26228** exists. The Indian Army is essentially saying: *"We know these mathematical defenses exist in academic papers, but we don't have a secure, automated software pipeline to actually use them in our daily operations."*

If you pitch tools like `BackdoorBench` or spectral signature detection, do not frame them as something the military already uses. Frame them as the **cutting-edge academic solution**

that your software will finally make possible for the military operations. You are building

write short report of repos which provides unique and bold solution with detail explanation give atleast 10 bold repos in report with explanation

This report outlines a complete architecture for a **Zero-Trust Machine Learning Supply Chain**, addressing the Ministry of Defence / Indian Army (DGIS) Problem Statement 26228. The solution is divided across the three core pillars of the AI lifecycle: Data Integrity, Model Tamper-proofing, and Inference Authenticity.

Here are 10 bold, open-source repositories to build this verifiable pipeline:

Pillar 1: Defending Against Data Poisoning & Tracking Provenance

To ensure training data has not been secretly compromised (e.g., backdoor triggers) and to track the exact origins of every file.

1. [SCLBD/BackdoorBench](#)

The Solution: A comprehensive evaluation and defense framework designed to detect adversarial vulnerability during the training stage. [GitHub](#)

Why it's bold: Rather than manually verifying data, this framework can act as an automated gatekeeper. It implements state-of-the-art defense algorithms (like Anti-Backdoor Learning or Spectral Signatures) to detect statistical anomalies caused by poisoned data before it enters the military pipeline.

2. [iterative/dvc](#) [Wikipedia](#)

The Solution: Data Version Control (DVC) creates immutable, cryptographically hashed snapshots of massive datasets.

Why it's bold: DVC calculates the MD5 hash of your raw training data and creates a pointer file tracked by Git. If an adversary alters even a single pixel in a dataset

of millions, the hash breaks, immediately alerting command to unauthorized data tampering. [DagsHub](#)

3. [in-toto/in-toto](#)

The Solution: A framework to protect software supply chain integrity by creating cryptographic attestations. [GitHub](#)

Why it's bold: [in-toto](#) ensures that every step (e.g., data scraping, cleaning, training) was performed by the authorized person/machine and hasn't been intercepted. If a script modifies data outside this attested pipeline, the chain fails.

4. [slsa-framework/slsa-github-generator](#) [GitHub](#)

The Solution: Generates non-forgeable Supply-chain Levels for Software Artifacts (SLSA) Level 3 provenance. [DevOps Cube](#)

Why it's bold: It uses an isolated, ephemeral build system to generate cryptographic receipts proving exactly how a model was compiled. It proves to the DoD that the model wasn't swapped out with a malicious clone during the build process. [DevOps Cube](#)

Pillar 2: Verifying Model Integrity (Tamper-Proofing)

To guarantee the model's weights remain exactly as the DoD trained them, defending against insider threats and post-training compromise.

1. [sigstore/model-transparency](#)

The Solution: Brings software supply chain signing to massive AI models.

Why it's bold: Standard cryptographic signing tools often fail on 10GB+ neural networks. This tool hashes and signs ML models component-by-component, binding the signature to a public (or private military) append-only transparency log (Rekor).

2. [sigstore/model-validation-operator](#)

The Solution: A Kubernetes-native operator for strict deployment verification.

Why it's bold: You can configure military edge clusters to physically *refuse* to load a model into memory unless its Sigstore cryptographic signature is verified in real-time. It completely automates the enforcement of model integrity at the deployment level.

3. [Trusted-AI/adversarial-robustness-toolbox](#) [GitHub](#)

The Solution: IBM's Python library (ART) for machine learning security. [GitHub](#)

Why it's bold: ART provides 39 attack modules and 29 defense modules. It allows you to actively "Red Team" the model against Evasion, Poisoning, Extraction, and Inference attacks, proving mathematically to the judges that the final model is physically robust against adversarial noise and tampering.

[adversarial-robustness-to...](#)

Pillar 3: Verifiable Inference & Output Authenticity

To prove that a deployed drone/edge device actually ran the right model, and that a "man-in-the-middle" didn't alter the classification output.

1. [zkonduit/ezkl](#) [GitHub](#)

The Solution: An engine for doing inference on deep learning models within a zk-SNARK (Zero-Knowledge Machine Learning). [GitHub](#)

Why it's bold: [ezkl](#) converts standard `.onnx` neural networks into a cryptographic circuit. When the model makes a prediction, it generates a lightweight zk-SNARK proof. This mathematically guarantees: *"I ran this exact neural network on this specific input, and it produced this exact output"*. The proof cannot be faked, even if the enemy hacks the device's operating system.

2. [mithril-security/blindai](#) [Microsoft Community Hub](#)

The Solution: An open-source AI deployment solution that leverages secure enclaves (Trusted Execution Environments like Intel SGX).

[Microsoft Community Hub](#)

Why it's bold: Even if an attacker gains root access to the machine running the AI, they cannot see or modify the model weights or the data being processed. The data is only decrypted inside a hardware-protected enclave, guaranteeing pristine, untampered inference. [Microsoft Community Hub](#)

3. [contentauth/c2pa-rs](#) [GitHub](#)

The Solution: The Rust SDK for the Coalition for Content Provenance and Authenticity (C2PA). [GitHub](#)

Why it's bold: Once the AI generates an output (e.g., an image with a bounding box around a target), [c2pa-rs](#) securely embeds a cryptographic signature directly into the file's metadata. This creates a verifiable "paper trail" proving the

give it in docs format to share it with my team

I have generated the report and saved it as a Google Doc so you can easily share it with your team.

Zero-Trust ML Supply Chain Architecture (SIH 26228)



Zero-Trust ML Suppl...

Open

Google Docs

give clickable links in docs to visit repo

I have created a new version of the Google Doc for you and your team that includes the clickable links directly to all the repositories.

Zero-Trust ML Supply Chain Architecture (with Links)



Zero-Trust ML Suppl...

Open

Google Docs

Alternatively, if you want to copy and paste them into your existing document, here are the direct links corresponding to the report:

Pillar 1: Data Poisoning & Provenance

1. [SCLBD/BackdoorBench](#)
2. [iterative/dvc](#)
3. [in-toto/in-toto](#)
4. [slsa-framework/slsa-github-generator](#)

Pillar 2: Model Integrity (Tamper-Proofing)

5. sigstore/model-transparency

6. sigstore/model-validation-operator

7. Trusted-AI/adversarial-robustness-toolbox

Pillar 3: Verifiable Inference & Output Authenticity

8. zkonduit/ezkl

9. mithril-security/blindai

10. contentauth/c2pa-rs